



Lasso and Ridge regression

Milica Maričić, Phd



Agenda

- Ridge regression
- Lasso regression
- Elastic net model





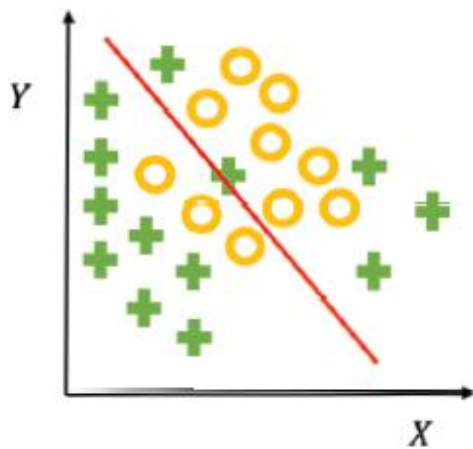
Linear regression model regularisation

- Linear regression models are practical because they are understandable, relatively easy to implement and very simple, considering that they are based on fitting a regression line (that is, a regression plane or hyperplane) through the data.
- If the model is statistically significant it can be used for prediction and estimation – Does it always work??
- No – some times the model is overfitted to the sampled data, and cannot handle new instances

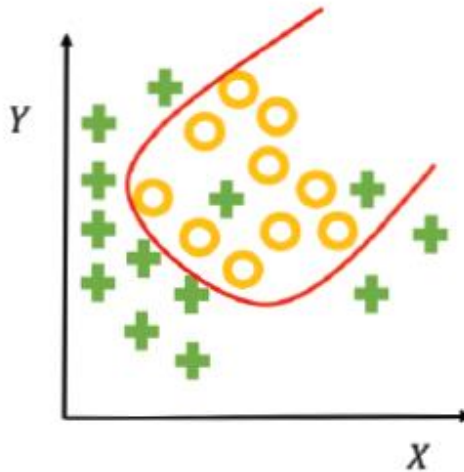




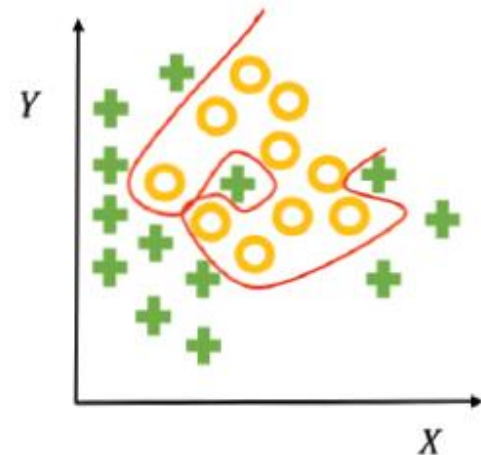
Linear regression model regularisation



Under-fitting



Appropriate-fitting



Over-fitting





Linear regression model regularisation

- What leads to over-fitting?
 - Outliers
 - Small sample, large number of predictors
 - Training data is not adequate
 - Predictors which are not correlated with the dependent variable
- Solution: **Regularisation**





Regularisation – Basic idea

- Idea: Reduce over-fitting, increase model reliability and prediction and estimation accuracy
- In LR, model complexity can lead to complex model interpretation. At the same time, model simplicity can lead to biased models.
- Solution: **Try to find a balance between bias and variance**
- **Regularisation aims at introducing restrictions on regression coefficients – it reduces the values of regression coefficients**





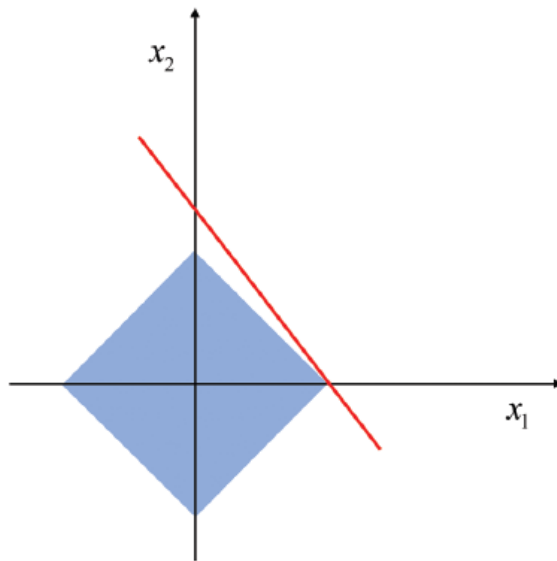
Regularisation – Basic idea

- **Regularisation: The simplest, the best**
- Two basic LR model constraints:
 - **L1-penalty:** The sum of the absolute values of the regression parameter estimates should be smaller than some constant
 - **L2-penalty:** The sum of the squares of the regression parameter scores should be smaller than some constant

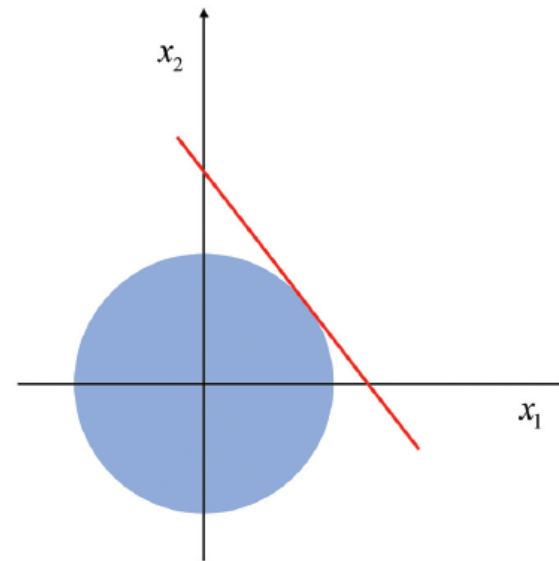




Regularisation – L1 and L2 constraints



L_1 - Penalty



L_2 - Penalty





Ridge regression

- Constraint: L2 – penalty

- OLS:
$$Q = \sum_{i=1}^N e_i^2 = \sum_{i=1}^N (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^N \left(Y_i - b_0 - \sum_{j=1}^k b_j X_{ij} \right)^2$$

- Ridge regression:
$$Q^R = \sum_{i=1}^N \left(Y_i - b_0 - \sum_{j=1}^k b_j X_{ij} \right)^2 + \lambda \sum_{j=1}^k b_j^2$$





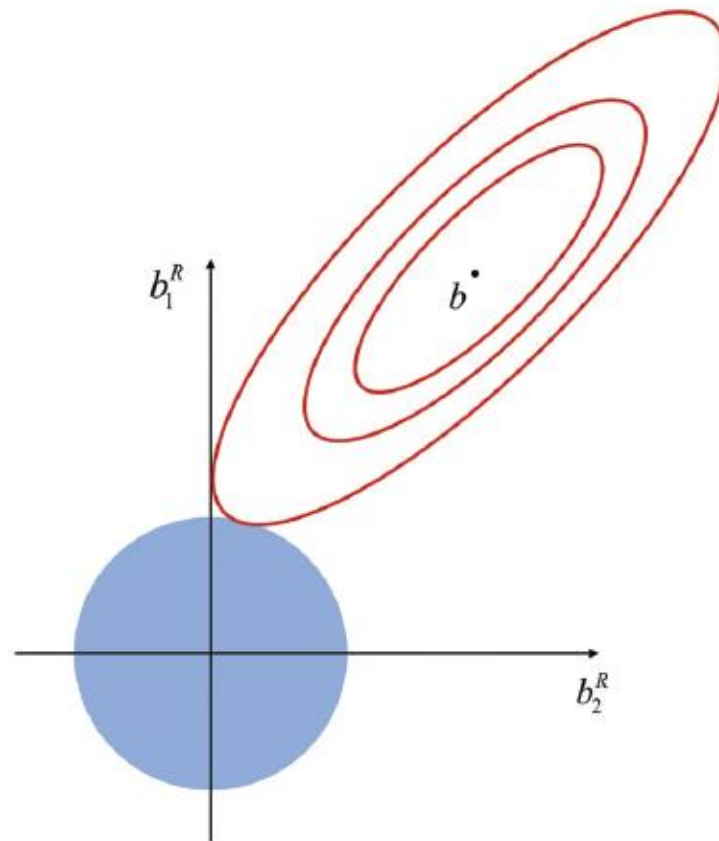
Ridge regression

- Parameter Lambda
 - Must be greater or equal to 0
 - Must be defined before conducting the analysis
- The second part of the equation is called the Shrinkage penalty
- We want to minimise it, so we want to reduce the regression coefficients to 0
- If $\text{Lambda}=0$, the Ridge regression is equal to LR
- If Lambda tends to infinity, the regression coefficients are close to zero



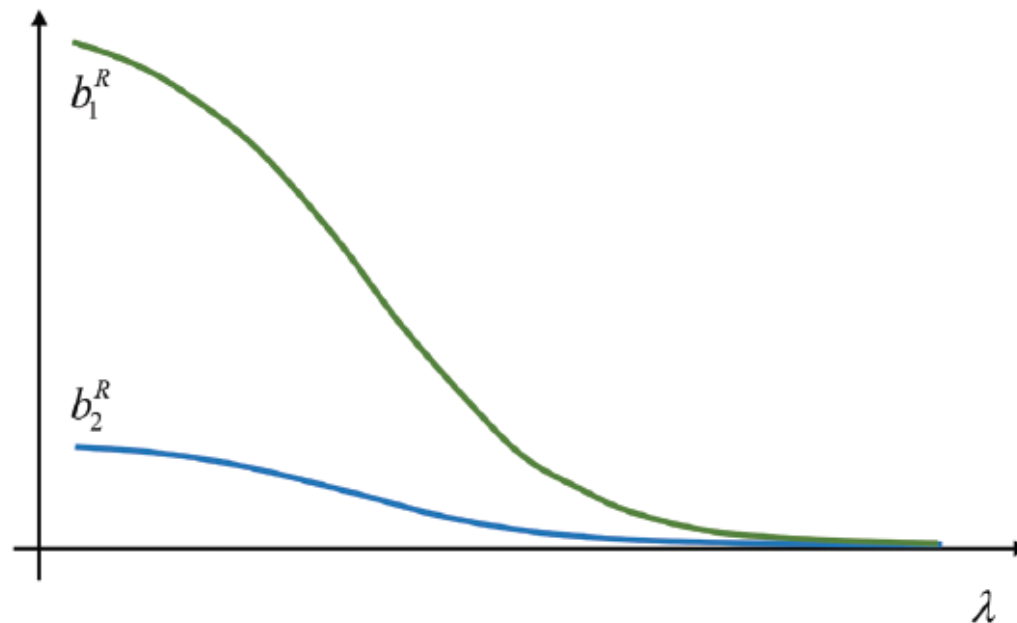


Graphical interpretation



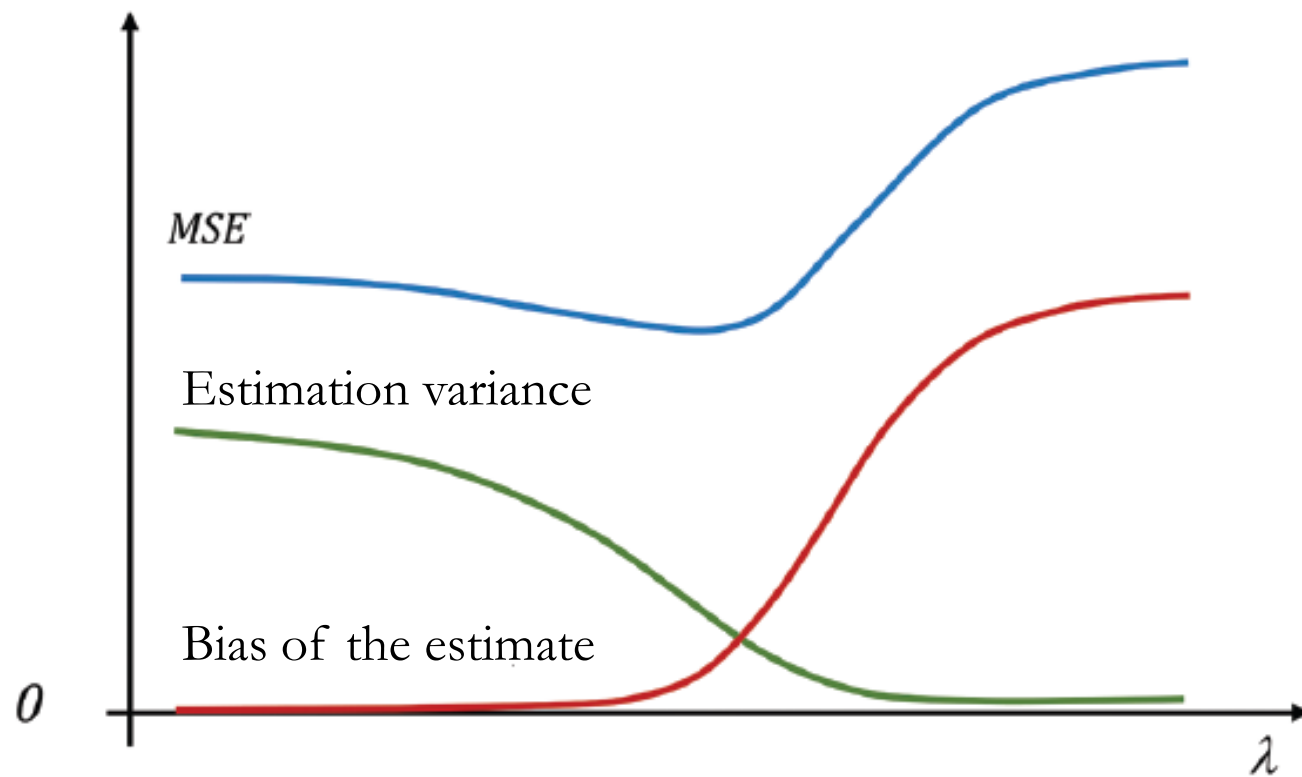


Graphical interpretation – impact of Lambda





How to determine Lambda?





Ridge regression

- Ridge regression cannot be done in SPSS, whereas it can be done in other statistical software such as R or Python
- Before we create and estimate a Ridge regression model, we will create a Linear regression model.
- We will do so in order to later compare the two models.
- We will observe the WEF.sav data set





Model 1

We are observing the WEF.sav matrix. Create and test the linear regression model of the influence of the variables *Market_size_Score_2013* and *Goods_market_efficiency_Score_2013* on the variable *Macroeconomic_environment_score_2013* through the following requests:

- I) Evaluate the parameters of the linear regression model using the method of least squares and give a graphical representation of the regression line;
- II) Examine the quality of representativeness of the regression line;
- III) Test the significance of the individual influence of the explanatory variable using the T-test and the significance of the entire regression model using the F-test, with a significance level of 0.05. Is the constant statistically significant in this model?;





Installing packages

```
# Install the packages needed to conduct linear regression
```

```
install.packages("foreign")
```

```
install.packages("ggplot2")
```

```
# Activate the packages needed to conduct linear regression
```

```
library(foreign)
```

```
library(ggplot2)
```

```
# Import the specific file
```

```
WEF <- read.spss("E:/Milica/Desktop/WEF.sav", to.data.frame = TRUE)
```

```
# Get to know your data
```

```
str(WEF)
```





Linear regression in R

```
# Create the LRM
regmodel1 <- lm(Macroeconomic_environment_score_2013 ~
Market_size_Score_2013 + Goods_market_efficiency_Score_2013, data
= WEF)

# List the results of the created LRM
summary(regmodel1)
```





Linear regression in R

```
call:
lm(formula = Macroeconomic_environment_score_2013 ~ Market_size_score_2013 +
    Goods_market_efficiency_score_2013, data = WEF)
```

```
Residuals:
    Min       1Q   Median       3Q      Max
-2.15180 -0.54376 -0.01665  0.46978  2.51433
```

```
Coefficients:
```

	Estimate	Std. Error	t value	Pr(> t)	
(Intercept)	1.60421	0.58487	2.743	0.00692	**
Market_size_score_2013	0.13472	0.06391	2.108	0.03690	*
Goods_market_efficiency_score_2013	0.60651	0.14347	4.227	4.35e-05	***

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Residual standard error: 0.8424 on 134 degrees of freedom
Multiple R-squared:  0.1943,    Adjusted R-squared:  0.1823
F-statistic: 16.16 on 2 and 134 DF,  p-value: 5.18e-07
```





Ridge regression in R

Install the needed package

```
install.packages("readr")
```

```
install.packages("glmnet")
```

Activate the needed packages

```
library(readr)
```

```
library(glmnet)
```





Ridge regression in R

```
# Import the data
```

```
WEF <- read_csv("D://Milica/WEF.csv")
```

```
# Create a subset with the independent variables
```

```
x1 <- as.matrix(data.frame(WEF$Market_size_Score_2013,  
  WEF$Goods_market_efficiency_Score_2013))
```





Ridge regression in R

```
# Create and conduct a ridge regression model
```

```
ridge1 <- glmnet(x1, WEF$Macroeconomic_environment_Score_2013, family  
= "gaussian", alpha = 0)
```

```
# Print some results
```

```
summary(ridge1)
```

```
print(ridge1)
```

Are these results useful?





Ridge regression in R

```
# Provide insights on a model for a particular Lambda  
coef(ridge1, s = 0.01)
```

```
> coef(ridge1, s = 0.01)  
3 x 1 sparse Matrix of class "dgCMatrix"  
  
              s1  
(Intercept)  1.7079940  
WEF.Market_size_Score_2013  0.1327209  
WEF.Goods_market_efficiency_Score_2013 0.5841478
```

Is this the best lambda? NO





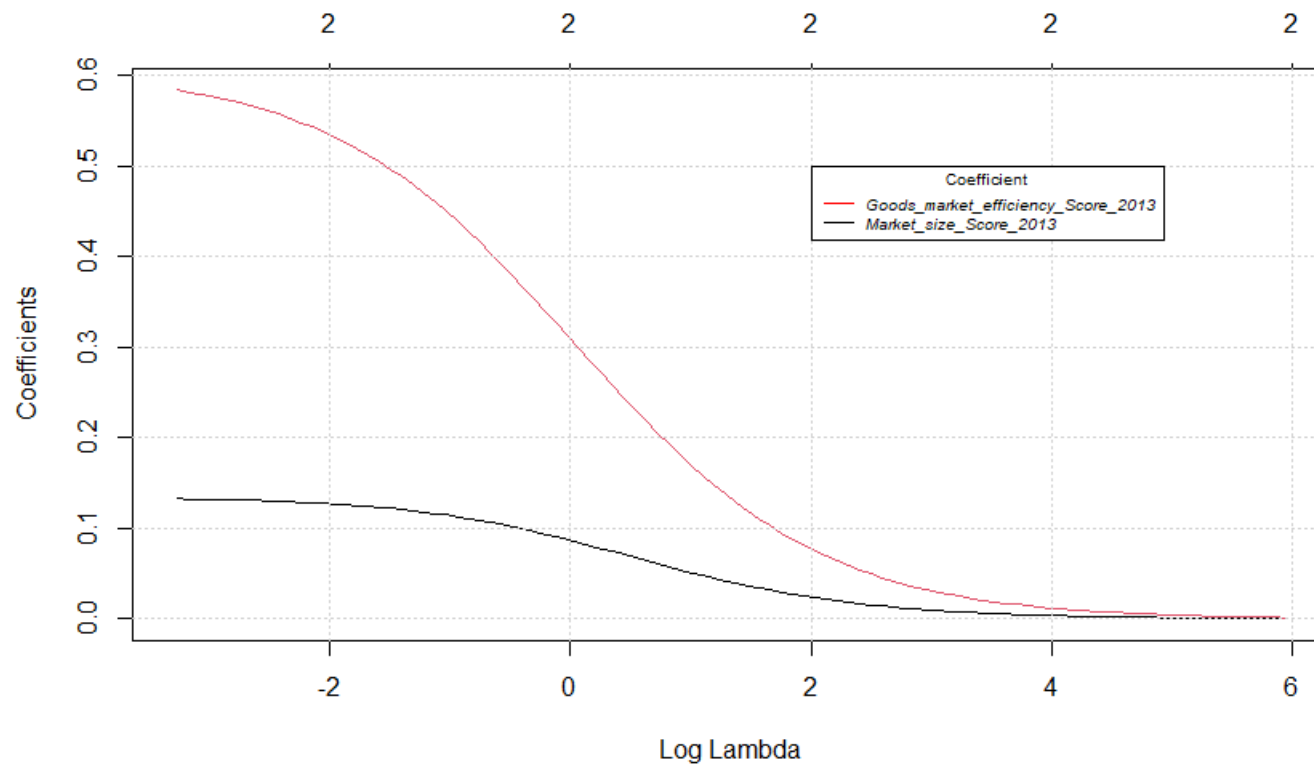
Ridge regression in R

```
# Change of coefficients as Lambda changes
plot(ridge1, xvar = "lambda")
grid()
legend(6, 0.75, legend = c("Goods_market_efficiency_Score_2013",
"Market_size_Score_2013"), col = c("red", "black"), lty = 1:1, cex =
0.6, title = "Coefficient", text.font = 4)
```





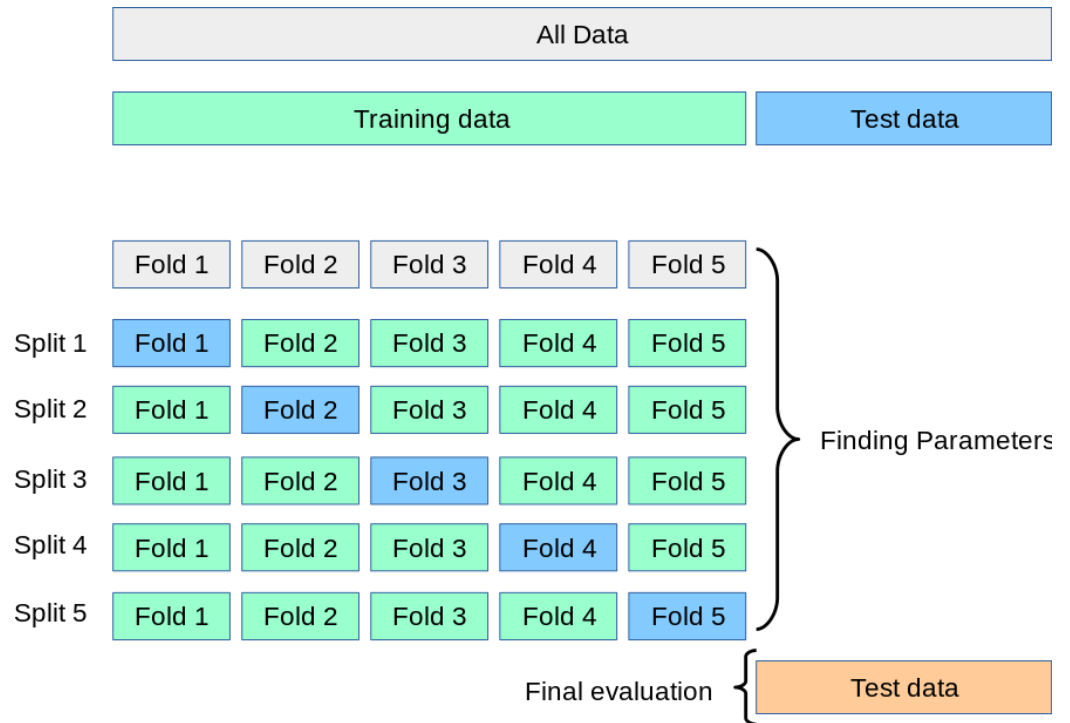
Ridge regression in R





Ridge regression in R

- As we do not know the best value of Lambda, we will use cross-validation
- K-fold cross-validation





Ridge regression in R

```
# Set seed of random generator
```

```
set.seed(123)
```

```
# Conduct ridge regression with cross validation
```

```
cv.ridge1 <- cv.glmnet(x1, WEF$Macroeconomic_environment_Score_2013,  
family = "gaussian", type.measure = "mse", alpha = 0)
```

```
# Print the results
```

```
print(cv.ridge1)
```





Ridge regression in R

- After 10-fold cross validation, we obtain the following:

Measure: Mean-Squared Error

	Lambda	Index	Measure	SE	Nonzero
min	0.066	94	0.7322	0.08526	2
1se	3.304	52	0.8150	0.06261	2

- From this output, we found that the best lambda is 0.066
- Next, we will use it to find the final ridge regression model

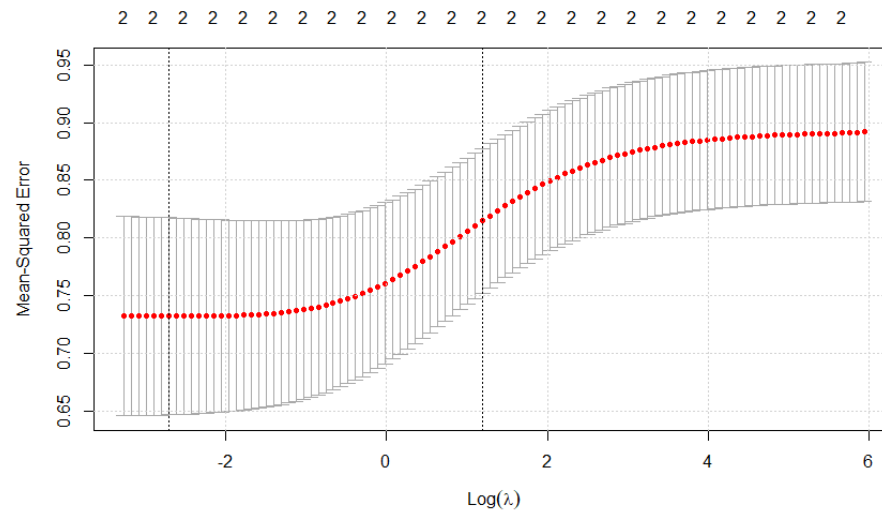




Ridge regression in R

- How MSE changes as lambda change?

```
# MSE change plot  
plot(cv.ridge1)  
grid()
```





Ridge regression in R

- Show and save best lambda

```
# Save best lambda as an object  
bestlambda1 <- cv.ridge1$lambda.min
```

```
# Print best lambda  
bestlambda1
```

```
> bestlambda1  
[1] 0.06639498
```





Ridge regression in R

```
# Create the final ridge regression model
```

```
ridge1best <- glmnet(x1, WEF$Macroeconomic_environment_Score_2013,  
family = "gaussian", alpha = 0, lambda = bestlambda1)
```

```
# Print model results
```

```
coef(ridge1best)
```

```
3 x 1 sparse Matrix of class "dgCMatrix"
```

	s0
(Intercept)	1.7808142
WEF.Market_size_Score_2013	0.1311626
WEF.Goods_market_efficiency_score_2013	0.5685927





Ridge regression in R

- The final ridge regression model is:

```
3 x 1 sparse Matrix of class "dgCMatrix"
               s0
(Intercept)    1.7808142
WEF.Market_size_Score_2013 0.1311626
WEF.Goods_market_efficiency_Score_2013 0.5685927
```

- The LR model was:

Coefficients:

(Intercept)	Market_size_Score_2013	Goods_market_efficiency_Score_2013
1.6042	0.1347	0.6065





Lasso regression

- Constraint: L1 – penalty

- OLS:
$$Q = \sum_{i=1}^N e_i^2 = \sum_{i=1}^N (Y_i - \hat{Y}_i)^2 = \sum_{i=1}^N \left(Y_i - b_0 - \sum_{j=1}^k b_j X_{ij} \right)^2$$

- Lasso regression:

$$Q^L = \sum_{i=1}^N \left(Y_i - b_0 - \sum_{j=1}^k b_j X_{ij} \right)^2 + \lambda \sum_{j=1}^k |b_j|$$





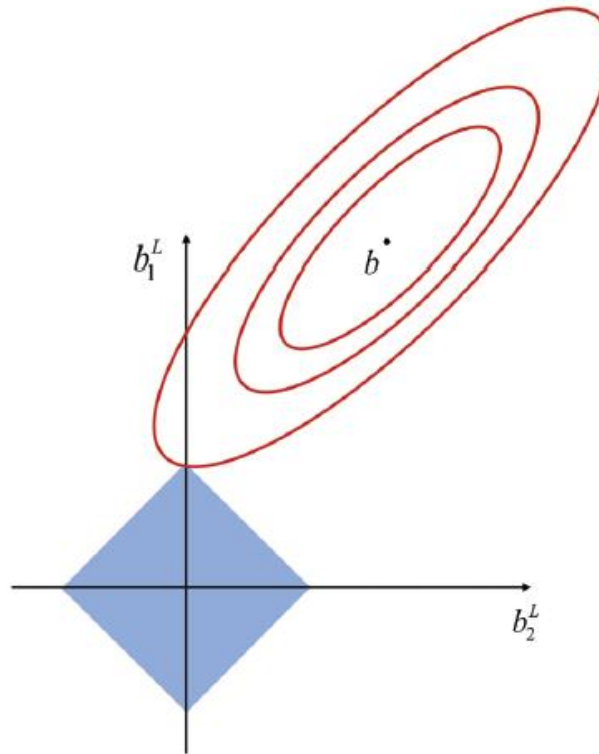
Lasso regression

- Parameter Lambda
 - Must be greater or equal to 0
 - Must be defined before conducting the analysis
- The second part of the equation is called the Shrinkage penalty
- We want to minimise it, so we want to reduce the regression coefficients to 0
- If $\text{Lambda}=0$, the Lasso regression is equal to LR
- If Lambda tends to infinity, the regression coefficients **are** zero



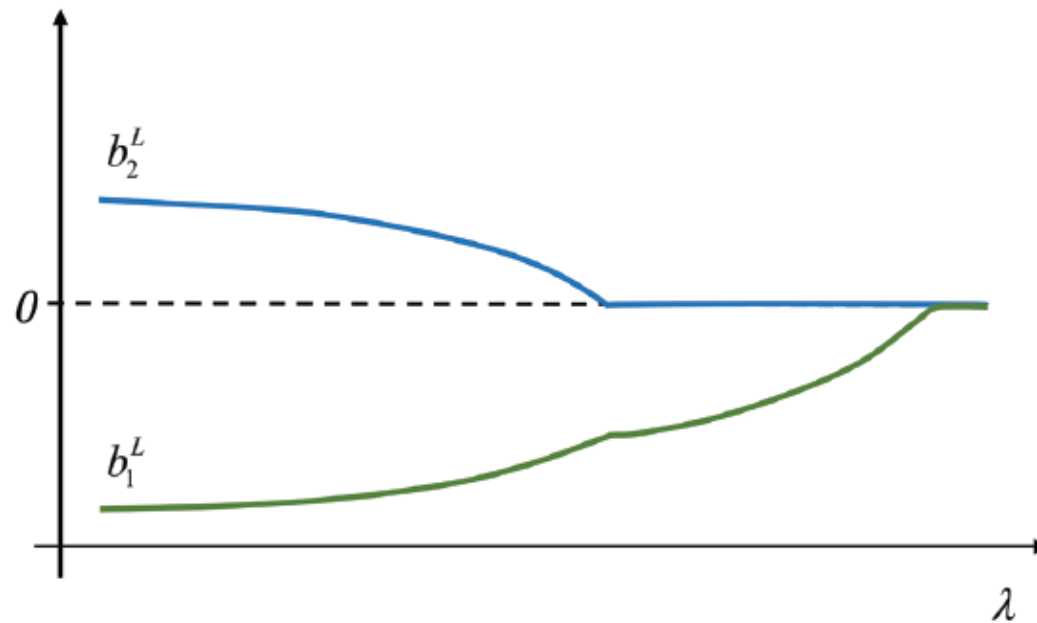


Graphical interpretation





Graphical interpretation – impact of Lambda





Ridge vs Lasso regression

- Lasso creates a subsample of independent variables
- Prediction accuracy of Ridge and Lasso regression are almost the same
- The bias level of the two regressions are the same
- Ridge variance is smaller than the Lasso variance
- There is no perfect answer which of the two is better – Maybe create a combination?





Ridge vs Lasso regression

- Lasso + Ridge = Elastic net

$$Q^{EN} = \sum_{i=1}^N \left(Y_i - b_0 - \sum_{j=1}^k b_j X_{ij} \right)^2 + \frac{(1-\alpha)}{2} \lambda \sum_{j=1}^k b_j^2 + \alpha \lambda \sum_{j=1}^k |b_j|$$

- Parameter alpha – balance between Lasso and Ridge
- If alpha=1 the model turns to Lasso regression
- If alpha=0 the model turns to Ridge regression





Lasso regression

- Lasso regression cannot be done in SPSS, whereas it can be done in other statistical software such as R or Python
- Before we create and estimate a Lasso regression model, we will create a Linear regression model.
- We will do so in order to later compare the two models.
- We will observe the WEF.sav data set and the same model as in previous example





Lasso regression in R

```
# Create and conduct a lasso regression model
```

```
lasso1 <- glmnet(x1, WEF$Macroeconomic_environment_Score_2013, family  
= "gaussian", alpha = 1)
```

```
# Print some results
```

```
summary(lasso1)
```

```
print(lasso1)
```

Are these results useful?





Lasso regression in R

```
# Provide insights on a model for a particular Lambda  
coef(lasso1, s = 0.01)
```

```
> coef(lasso1, s = 0.01)  
3 x 1 sparse Matrix of class "dgCMatrix"  
  
              s1  
(Intercept) 1.6873396  
WEF.Market_size_Score_2013 0.1285102  
WEF.Goods_market_efficiency_score_2013 0.5926112
```

Is this the best lambda? NO





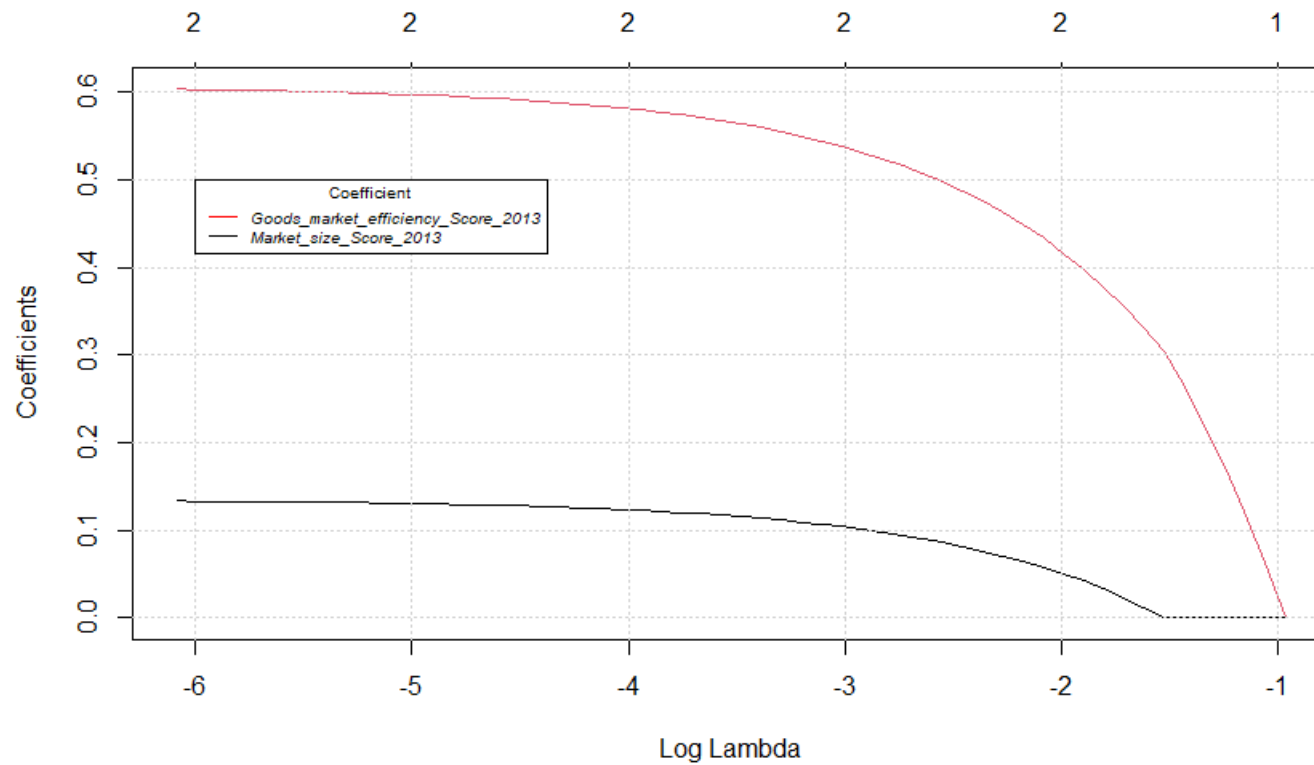
Lasso regression in R

```
# Change of coefficients as Lambda changes
plot(lasso1, xvar = "lambda")
grid()
legend(6, 0.75, legend = c("Goods_market_efficiency_Score_2013",
"Market_size_Score_2013"), col = c("red", "black"), lty = 1:1, cex =
0.6, title = "Coefficient", text.font = 4)
```





Lasso regression in R





Lasso regression in R

```
# Set seed of random generator
```

```
set.seed(123)
```

```
# Conduct lasso regression with cross validation
```

```
cv.lasso1 <- cv.glmnet(x1, WEF$Macroeconomic_environment_Score_2013,  
family = "gaussian", type.measure = "mse", alpha = 1)
```

```
# Print the results
```

```
print(cv.lasso1)
```





Lasso regression in R

- After 10-fold cross validation, we obtain the following:

Measure: Mean-Squared Error

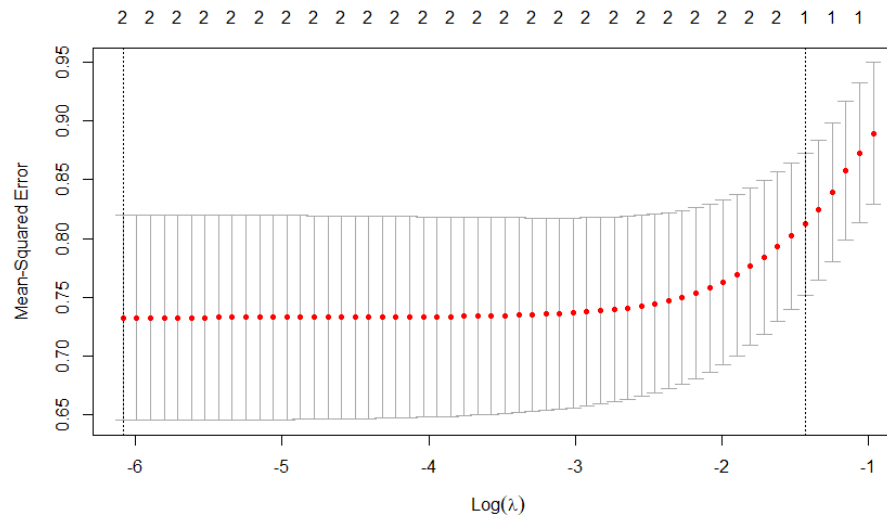
	Lambda	Index	Measure	SE	Nonzero
min	0.00228	56	0.7328	0.08743	2
1se	0.23861	6	0.8122	0.06062	1

- From this output, we found that the best lambda is 0.00228
- Next, we will use it to find the final lasso regression model





```
# MSE change plot
plot(cv.lasso1)
grid()
```





Lasso regression in R

- Show and save best lambda

```
# Save best lambda as an object  
bestlambda1 <- cv.lasso1$lambda.min
```

```
# Print best lambda  
bestlambda1
```

```
> bestlambda1  
[1] 0.002277659
```





Lasso regression in R

```
# Create the final ridge regression model
```

```
lassolbest <- glmnet(x1, WEF$Macroeconomic_environment_Score_2013,  
family = "gaussian", alpha = 1, lambda = bestlambda1)
```

```
# Print model results
```

```
coef(lassolbest)
```

```
> coef(lassolbest)  
3 x 1 sparse Matrix of class "dgCMatrix"  
  
              s0  
(Intercept)  1.6231402  
WEF.Market_size_Score_2013  0.1333062  
WEF.Goods_market_efficiency_Score_2013 0.6033447
```





Lasso regression in R

- The final lasso regression model is:

```
3 x 1 sparse Matrix of class "dgCMatrix"
                                     s0
(Intercept)                        1.6231402
WEF.Market_size_Score_2013         0.1333062
WEF.Goods_market_efficiency_Score_2013 0.6033447
```

- The final ridge regression model is:

```
3 x 1 sparse Matrix of class "dgCMatrix"
                                     s0
(Intercept)                        1.7808142
WEF.Market_size_Score_2013         0.1311626
WEF.Goods_market_efficiency_Score_2013 0.5685927
```

- The LR model was:

```
Coefficients:
               (Intercept)      Market_size_Score_2013  Goods_market_efficiency_Score_2013
               1.6042                0.1347                0.6065
```





Elastic net model in R

Install the needed package

```
install.packages("carret")
```

Activate the needed packages

```
library(carret)
```





Elastic net model in R

```
# Set seed
```

```
set.seed(42)
```

```
# Define the cross-validation which will be conducted
```

```
cv_5 = trainControl(method = "cv", number = 5)
```





Elastic net model in R

```
# Create the models
```

```
hit_elnet = train(  
  Macroeconomic_environment_Score_2013 ~ Market_size_Score_2013 +  
    Goods_market_efficiency_Score_2013, data = WEF,  
  method = "glmnet",  
  trControl = cv_5  
)
```

```
# Print the results
```

```
hit_elnet
```





Elastic net model in R

```
# Create the models
```

```
hit_elnet = train(  
  Macroeconomic_environment_Score_2013 ~ Market_size_Score_2013 +  
    Goods_market_efficiency_Score_2013, data = WEF,  
  method = "glmnet",  
  trControl = cv_5  
)
```

```
# Print the results
```

```
hit_elnet
```





Elastic net model in R

```
glmnet
```

```
137 samples  
2 predictor
```

```
No pre-processing
```

```
Resampling: Cross-validated (5 fold)
```

```
Summary of sample sizes: 109, 110, 110, 110, 109
```

```
Resampling results across tuning parameters:
```

alpha	lambda	RMSE	Rsquared	MAE
0.10	0.0007598729	0.8568842	0.1927678	0.6670697
0.10	0.0075987294	0.8568631	0.1927865	0.6670834
0.10	0.0759872943	0.8555481	0.1933548	0.6676915
0.55	0.0007598729	0.8570060	0.1925953	0.6672012
0.55	0.0075987294	0.8570068	0.1924529	0.6675050
0.55	0.0759872943	0.8582377	0.1896852	0.6727085
1.00	0.0007598729	0.8570604	0.1925353	0.6672537
1.00	0.0075987294	0.8571650	0.1921071	0.6679303
1.00	0.0759872943	0.8623629	0.1847828	0.6795387

```
RMSE was used to select the optimal model using the smallest value.
```

```
The final values used for the model were alpha = 0.1 and lambda = 0.07598729.
```

```
. |
```





Elastic net model in R

```
# Create the models and expand the search space
```

```
hit_elnet_int = train(  
  Macroeconomic_environment_Score_2013 ~ Market_size_Score_2013 +  
    Goods_market_efficiency_Score_2013, data = WEF,  
  method = "glmnet",  
  trControl = cv_5,  
  tuneLength = 10  
)
```

```
# Print the results
```

```
hit_elnet_int
```





Elastic net model in R

```
glmnet
```

```
137 samples
 2 predictor
```

```
No pre-processing
```

```
Resampling: Cross-validated (5 fold)
```

```
Summary of sample sizes: 109, 110, 110, 109, 110
```

```
Resampling results across tuning parameters:
```

alpha	lambda	RMSE	Rsquared	MAE
0.1	0.0009368069	0.8443864	0.2016689	0.6565473
0.1	0.0021641454	0.8443864	0.2016689	0.6565473
0.1	0.0049994566	0.8443864	0.2016689	0.6565473
0.1	0.0115493931	0.8443798	0.2017195	0.6566806
0.1	0.0266805960	0.8443242	0.2018336	0.6570272
0.1	0.0616356371	0.8443776	0.2020668	0.6581993
0.1	0.1423863159	0.8452972	0.2024569	0.6614594
0.1	0.3289308573	0.8501106	0.2027806	0.6689197
0.2	0.0009368069	0.8444076	0.2016160	0.6565687
0.2	0.0021641454	0.8444076	0.2016160	0.6565687
0.2	0.0049994566	0.8444194	0.2016159	0.6565833
0.2	0.0115493931	0.8444532	0.2016054	0.6568306
0.2	0.0266805960	0.8445088	0.2015709	0.6573699
0.2	0.0616356371	0.8448832	0.2014578	0.6591715
0.2	0.1423863159	0.8468404	0.2010301	0.6637969
0.2	0.3289308573	0.8552364	0.1990982	0.6746202
0.3	0.0009368069	0.8444346	0.2015732	0.6566049

0.7	0.0266805960	0.8455614	0.2001563	0.6594033
0.7	0.0616356371	0.8481030	0.1978335	0.6646124
0.7	0.1423863159	0.8583111	0.1899203	0.6782428
0.7	0.3289308573	0.8885148	0.1893192	0.7091842
0.8	0.0009368069	0.8444954	0.2014579	0.6566635
0.8	0.0021641454	0.8444954	0.2014579	0.6566635
0.8	0.0049994566	0.8446401	0.2013166	0.6569878
0.8	0.0115493931	0.8449277	0.2008950	0.6577376
0.8	0.0266805960	0.8457986	0.1998517	0.6598740
0.8	0.0616356371	0.8488949	0.1969716	0.6657393
0.8	0.1423863159	0.8614516	0.1865560	0.6816653
0.8	0.3289308573	0.8962466	0.1948680	0.7157016
0.9	0.0009368069	0.8444808	0.2014552	0.6566470
0.9	0.0021641454	0.8444956	0.2014546	0.6566650
0.9	0.0049994566	0.8446728	0.2012656	0.6570534
0.9	0.0115493931	0.8450126	0.2007722	0.6578928
0.9	0.0266805960	0.8460449	0.1995394	0.6603562
0.9	0.0616356371	0.8497392	0.1960570	0.6668788
0.9	0.1423863159	0.8641100	0.1849838	0.6846454
0.9	0.3289308573	0.9048712	0.1987682	0.7223610
1.0	0.0009368069	0.8444850	0.2014528	0.6566558
1.0	0.0021641454	0.8445202	0.2014479	0.6566982
1.0	0.0049994566	0.8447057	0.2012143	0.6571192
1.0	0.0115493931	0.8450991	0.2006480	0.6580484
1.0	0.0266805960	0.8463004	0.1992193	0.6608407
1.0	0.0616356371	0.8506369	0.1950863	0.6680309
1.0	0.1423863159	0.8664810	0.1846906	0.6874120
1.0	0.3289308573	0.9151363	0.1611178	0.7300655

RMSE was used to select the optimal model using the smallest value.
The final values used for the model were alpha = 0.1 and lambda = 0.0266806.





Elastic net model in R

```
# Find the best combination of alpha and lambda
get_best_result = function(caret_fit) {
  best = which(rownames(caret_fit$results) ==
rownames(caret_fit$bestTune))
  best_result = caret_fit$results[best, ]
  rownames(best_result) = NULL
  best_result
}

# Print the best combination model evaluation
get_best_result(hit_elnet_int)
```





Elastic net model in R

```
> get_best_result(hit_elnet_int)
  alpha   lambda    RMSE Rsquared    MAE   RMSESD RsquaredSD    MAESD
1  0.1 0.0266806 0.8443242 0.2018336 0.6570272 0.1109204 0.1384262 0.09982563
> |
```





Lasso and Ridge regression

Milica Maričić, Phd